FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional
Carrera: Licenciatura en Ciberdefensa

Materia:
Lenguajes de programación

Profesor a cargo: Ing. Sebastián Balestrini

Año: 2025

Carga horaria: 64 hs

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Fundamentación:

En el ámbito de la tecnología, la ingeniería de software y, particularmente, en el desarrollo de soluciones informáticas para diversos sectores, el dominio de los lenguajes de programación constituye una competencia esencial para la formación de profesionales altamente capacitados. La creciente digitalización de procesos, la constante evolución de las herramientas y entornos de desarrollo, y la necesidad de implementar soluciones eficientes y seguras hacen que la programación sea un pilar fundamental en la industria tecnológica contemporánea.

Comprender y aplicar los principios de la programación permite a los futuros profesionales no solo desarrollar software, sino también abordar problemas de forma sistemática, optimizar recursos, automatizar procesos y garantizar la calidad de los productos desarrollados. La asignatura proporciona una base sólida en conceptos y estructuras comunes a múltiples lenguajes, fomentando la capacidad de adaptación a nuevas tecnologías y paradigmas de desarrollo que surgen en un mercado en constante transformación.

Es por ello que se esta Asignatura adquiere relevancia en el Plan de Estudios de acuerdo con:

1. En el campo de la Ciberdefensa, gran parte de la documentación técnica, repositorios de código, scripts de automatización, bibliotecas especializadas y guías de herramientas forenses están escritas en diversos lenguajes de programación. Contar con una base sólida en programación permite comprender, adaptar e implementar soluciones a partir de estos recursos, facilitando la respuesta a incidentes y la investigación de ciberamenazas.
2. Las tareas de análisis forense, monitoreo de redes, pruebas de penetración y hacking ético requieren el dominio de scripts, comandos y programas escritos en lenguajes como Python, C/C++, JavaScript, Bash o PowerShell. Esta asignatura proporciona los fundamentos que permiten interpretar, modificar y desarrollar código para optimizar el uso de estas herramientas en operaciones de ciberdefensa.
3. El desarrollo de software y la respuesta a incidentes de seguridad son actividades colaborativas que involucran el trabajo conjunto con otros

profesionales, muchas veces en contextos internacionales. El conocimiento de lenguajes de programación y sus principios facilita la comunicación técnica, la contribución a proyectos de código abierto y la integración en equipos multidisciplinarios.

4. Las certificaciones y el desarrollo profesional:
   - El saber programar otorga al estudiante de ventajas claras en seguridad como ser:
   - Comprender vulnerabilidades de código: Saber leer y entender código permite detectar fallos como SQL Injection, Cross-Site Scripting (XSS), buffer overflows, etc.
   - Automatizar tareas de seguridad: Muchos roles en seguridad implican crear scripts para escaneo, análisis o explotación, usando Python, Bash, PowerShell, etc.
   - Analítica forense y malware: En ciberforense y análisis de malware, leer código en C, Java, ensamblador o Python ayuda a entender cómo actúa un software malicioso.
   - Seguridad en desarrollo seguro (DevSecOps): Comprender la lógica de programación es clave para auditar código y recomendar buenas prácticas a los equipos de desarrollo.
   - Tanto en CEH como OSCP es muy beneficioso saber programar. escribir scripts y entender código vulnerable. Python, Bash y lenguaje de C son lenguajes muy necesarios.

5. La implementación de defensas proactivas, la automatización de tareas de seguridad, la creación de herramientas de análisis forense y la personalización de utilidades para el hacking ético dependen de la habilidad para diseñar algoritmos eficientes, manipular estructuras de datos y comprender el flujo lógico de un programa. La formación en lenguajes de programación brinda la base para innovar y adaptarse a la constante evolución de las ciberamenazas.

Objetivos

Generales:

Que el estudiante logre:

- Alcanzar una comprensión sólida de los fundamentos de la programación, permitiéndoles interpretar, diseñar y desarrollar soluciones en distintos lenguajes de forma eficiente y segura.

- Aplicar los conceptos de programación para la automatización de tareas, el análisis de datos y la creación de herramientas específicas orientadas a la ciberdefensa, el análisis forense y el hacking ético.
- Desarrollar el pensamiento lógico y la capacidad de abstracción necesarios para modelar problemas y diseñar algoritmos que optimicen recursos y procesos en entornos de seguridad informática.
- Adaptarse a nuevas tecnologías y lenguajes de programación, brindando el marco teórico y las habilidades prácticas que faciliten el aprendizaje continuo y la incorporación de herramientas emergentes.
- Desarrollar competencias para trabajar de forma colaborativa en equipos multidisciplinarios, interpretando y produciendo código que pueda integrarse en proyectos conjuntos, tanto a nivel local como internacional.

Específicos:

Que el estudiante logre:

- Identificar los conceptos fundamentales de la programación y su terminología, diferenciando paradigmas y estructuras comunes a distintos lenguajes.
- Reconocer los diferentes tipos y niveles de lenguajes de programación, así como su aplicabilidad en el desarrollo de soluciones para la ciberdefensa.
- Comprender el concepto de algoritmo y aplicar metodologías para su diseño, representación y análisis.
- Desarrollar programas simples utilizando estructuras de control, tipos de datos y funciones, garantizando claridad, legibilidad y eficiencia.
- Implementar estructuras de datos básicas para almacenar, procesar y recuperar información de manera eficiente.
- Aplicar principios de programación orientada a objetos para modelar problemas y construir soluciones modulares y reutilizables.
- Utilizar PHP y conceptos de programación web para la generación de herramientas de análisis y automatización de procesos en entornos de Ciberdefensa.
- Emplear herramientas y entornos de desarrollo para escribir, depurar y mantener código en distintos lenguajes.
- Integrar scripts y fragmentos de código en procesos de análisis forense digital y pruebas de penetración, interpretando y adaptando soluciones existentes.
- Desarrollar la capacidad de trabajo colaborativo mediante el uso de control de versiones y la participación en proyectos de programación compartidos.

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Contenidos

Unidad 1. Introducción a la programación, algoritmos y diagramas de flujo

Objetivos:

- Comprender qué es programar y qué es un algoritmo.
- Reconocer tipos y niveles de lenguajes de programación.
- Entender las fases del desarrollo de software.
- Representar procesos mediante diagramas de flujo.

Contenidos:

- ¿Qué es programar? ¿Qué es un algoritmo?
- Lenguajes de programación: máquina, bajo nivel, alto nivel.
- Lenguajes compilados vs. interpretados.
- Estilos de programación: lineal, estructurada, modular, orientada a objetos.
- Proceso de desarrollo de software: análisis, diseño, codificación, prueba, documentación, mantenimiento.
- Diagramas de flujo

Unidad 2. Pseudocódigo – estructuras secuenciales, condicionales, múltiples y repetitivas

Objetivos:

- Aprender a escribir algoritmos en pseudocódigo.
- Aplicar estructuras de control (decisión y repetición).
- Consolidar la lógica algorítmica antes de programar en C

Contenidos:

- Sintaxis del pseudocódigo
- Variables y constantes
- Estructuras de control
- Operadores

Unidad 3. Iniciación en C – del pseudocódigo a código

Objetivos:

- Traducir pseudocódigo a código en lenguaje C.

- Aplicar en C lo aprendido en estructuras de control.
- Familiarizarse con la estructura básica de un programa en C.

Contenidos:

- Inclusión de librerías (#include)
- main() , printf() , scanf()
- Declaración de variables y tipos de datos (int, float, char)
- Estructuras de control en C
- Comparación entre pseudocódigo y C

Unidad 4. Algoritmos básicos y arreglos

Objetivos:

- Trabajar con vectores y matrices.
- Aplicar búsquedas simples y ordenamientos básicos.

Contenidos:

- Vectores y matrices
- Búsqueda lineal
- Ordenamiento burbuja básico
- Recorridos con for

Unidad 5. Funciones y modularización

Objetivos:

- Entender la división del programa en funciones.
- Aplicar paso por valor y referencia.
- Organizar el código de forma clara y escalable.

Contenidos:

- Definición de funciones
- Reglas de Ámbito
- Variables locales y globales
- Parámetros
- Buenas prácticas

Unidad 6. Comparativa entre lenguajes clásicos

Objetivos:

- Reconocer diferencias y similitudes entre distintos lenguajes.
- Comparar estilos y estructuras básicas.

Contenidos:

- Lenguajes: C, Pascal, QuickBasic, COBOL, Assembler
- Entrada/salida, condicionales, bucles, funciones en cada lenguaje

Unidad 7. Introducción al paradigma orientado a objetos

Objetivos:

- Comprender en qué se diferencia la POO del paradigma estructurado.
- Conocer los conceptos básicos de clase, objeto, atributos y métodos.
- Identificar la relación entre modelo del mundo real y objetos en programación.

Contenidos:

- Paradigmas de programación: estructurado vs. orientado a objetos.
- Concepto de clase y objeto.
- Atributos (estado) y métodos (comportamiento).
- Instanciación de objetos.
- Ejemplos simples en C++ y Java

Unidad 8. Principios fundamentales de la POO

Objetivos:

- Comprender los principios de encapsulamiento, herencia y polimorfismo.
- Analizar cómo estos principios ayudan a organizar y reutilizar código.
- Aprender a representar relaciones entre clases mediante un diagrama UML básico.
- Ver ejemplos básicos de estos conceptos en C++ y Java.

Contenidos:

- Encapsulamiento: modificadores de acceso (public, private, protected).
- Herencia: clases padre e hijo, reutilización de atributos y métodos.
- Polimorfismo: sobrecarga y sobrescritura
- Relación entre los conceptos y las ventajas de la POO.
- Introducción mínima a UML:
- Qué es un diagrama de clases UML y para qué sirve.

- Símbolos básicos: clases, atributos, métodos, herencia.
- Ejemplo UML
- Ejemplos simples en C++ y Java.

Unidad 9. Introducción a la Programación Web, al Lenguaje HTML y al lenguaje PHP

Objetivos:

- Comprender la arquitectura cliente-servidor y cómo funciona la web.
- Diferenciar entre lenguajes de marcado (HTML) y lenguajes de programación web.
- Conocer ejemplos de lenguajes web del lado del cliente y del lado del servidor.
- Introducir la sintaxis básica de PHP para generar contenido dinámico..

Contenidos:

- Arquitectura web: Cliente, servidor, protocolo HTTP, ciclo de petición/respuesta.
- Lenguajes de marcado vs. lenguajes de programación web
- Qué es HTML y para qué sirve.
- Ejemplos de lenguajes en programación web
- Diferencia entre lenguajes del lado del cliente y del lado del servidor
- Introducción a PHP

Unidad 10. Formularios HTML , Procesamiento del lado del servidor en PHP y lenguajes backend

Objetivos:

- Comprender cómo se envían y procesan datos desde un formulario HTML.
- Usar variables superglobales en PHP para recibir datos.
- Aplicar estructuras de control y funciones básicas en PHP.
- Conocer qué otros lenguajes y frameworks backend existen además de PHP
- Comprender posibles vulnerabilidades en la programación del lado del servidor

Contenidos:

- Formularios HTML y su interacción con PHP
- Estructuras de control y funciones en PHP
- Otros lenguajes backend

- Puntos de vulnerabilidad en programación web

Estrategias de enseñanza

La materia se organiza en sesiones semanales que se correlacionan con las unidades de contenidos de la materia. Las lecturas obligatorias serán acompañadas por diversos recursos digitales que se han desarrollado por los docentes de la cátedra.

Entre ellos se encuentran:

- Material didáctico de la Unidad temática
- Clases asíncronas en video, dando apoyo a la unidad temática
- Material de lectura adicional
- Foros de consultas y debates
- Actividades de autoevaluación
- Actividades prácticas para el alumno

Régimen de aprobación de la materia

La materia podrá ser aprobada con promoción directa o promoción con examen final. Para aprobar la materia por el régimen de promoción directa, los estudiantes deberán cumplir con los siguientes **requisitos:**

i) Cumplir la totalidad de las actividades obligatorias propuestas por el docente a cargo de cada asignatura.

ii) Aprobar al menos DOS (2) instancias de evaluación con una calificación no inferior a SIETE (7) puntos en todos los casos. En los casos en los que fuese necesario expresar el número entero del promedio de notas parciales, se aplicará el número entero superior si la fracción fuere de 0,50 puntos o más y el número entero inferior si fuere de 0,49 o menos; cuando la nota fuese de 3,01 a 3,99, se colocará 3 puntos.

iii) No contar con aplazos.

Para aprobar la materia por el régimen de promoción con examen final, los estudiantes deberán cumplir con los siguientes **requisitos:**

i) Cumplir la totalidad de las actividades obligatorias propuestas por el docente a cargo de cada asignatura.

ii) Aprobar al menos DOS (2) instancias de evaluación con una calificación no inferior a CUATRO (4) puntos en todos los casos.

iii) Aprobar un examen final presencial con una calificación igual o superior a CUATRO (4) puntos. En los casos en los que fuese necesario expresar el número entero del promedio de notas parciales, se aplicará el número entero superior si la fracción fuere de 0,50 puntos o más y el número entero inferior si fuere de 0,49 o menos; cuando la nota fuese de 3,01 a 3,99, se colocará 3 puntos. La calificación final corresponderá a la que se obtenga en el examen final. Los estudiantes dispondrán de TRES (3) turnos de examen final consecutivos a la finalización de la respectiva cursada. Transcurrido ese plazo, se perderá la regularidad en la asignatura y deberá recursar la unidad curricular.

Recuperatorio

El estudiante que no asista a un examen parcial y pueda acreditar mediante certificado médico la inasistencia, así como el que desapruebe el examen en la fecha originalmente prevista, podrá rendir un examen recuperatorio. La calificación obtenida en esta instancia reemplaza el ausente o el insuficiente, según corresponda. Sólo podrá presentarse al recuperatorio de 1 (UNA) instancia parcial.

Bibliografía

Obligatoria

No aplica

Complementaria

- Goin, M. (2016). Caminando junto al lenguaje C (Lecturas de Cátedra). Universidad Nacional de Río Negro. Licencia CC BY-NC-ND 2.5 Argentina.

- Urrutia Landa, G. (1999). Programación en C para principiantes. [Licencia Creative Commons Atribución-NoComercial-SinDerivadas 4.0 Internacional]

- Marzal Varó, A., & Gracia, I. (2010). Introducción a la programación con C. Universitat Jaume I, Servei de Comunicació i Publicacions. ISBN: 978-84-693-0143-2

- Stallman, R., Rothwell, T., & Beebe, N. (s.f.). GNU C language introduction and reference manual. Free Software Foundation.

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

- Bergmann, S. D. (2017). Introduction to Computer Science with Java Programming. Rowan University Press. DOI: 10.31986/issn.2689-0690_rdw.oer.1000

- Manual Oficial de PHP: https://www.php.net/manual/es/

- Manual de referencia de MySQL: https://dev.mysql.com/doc/refman/8.4/en/

- Raúl González Duque. Python para todos.

- Allen B. Downey (2015). Think Python. 2nd Edition.